

# Основы WEB программирования

Содержание

HTML и CSS

JavaScript

## HTML и CSS

HTML (HyperText Markup Language) — это стандартный язык разметки для создания веб-страниц. Он используется для структурирования и представления контента в интернете. HTML позволяет разработчикам определять заголовки, абзацы, ссылки, изображения и другие элементы, которые составляют веб-страницу. Понимание структуры HTML-документа является первым шагом к созданию собственных веб-страниц.

### Основной теоретический материал по HTML и CSS

Теоретический материал по HTML и CSS представлен на следующих ресурсах:

Самоучитель HTML ([abuzov.com](http://abuzov.com))

Руководство по HTML5 и CSS3 ([Metanit](http://metanit.com))

## JavaScript

JavaScript — это высокоуровневый, интерпретируемый язык программирования, который делает веб-страницы интерактивными. В отличие от HTML (который определяет структуру) и CSS (который отвечает за оформление), JavaScript добавляет поведение и динамику на веб-страницы.

### Основные характеристики JavaScript

**Клиентский язык** - выполняется в браузере пользователя

**Интерпретируемый** - не требует компиляции

**Мультипарадигменный** - поддерживает ООП, функциональное и императивное программирование

**Динамическая типизация** - типы данных определяются автоматически

**Поддержка асинхронного программирования** - с помощью колбэков, промисов и `async/await`

Для чего нужен JavaScript в веб-разработке?

**Интерактивность** - реакции на действия пользователя

(клики, наведение, ввод данных)

**Динамическое изменение контента** - обновление

страницы без перезагрузки

**Анимации и визуальные эффекты** - плавные переходы,

параллакс-эффекты

**Валидация форм** - проверка данных перед отправкой

**Работа с API** - получение данных с сервера без

перезагрузки страницы

**Хранение данных на клиенте** - localStorage,

sessionStorage, cookies

**Создание SPA (Single Page Applications)** - например,

React, Angular, Vue.js

**Содержание**

**HTML и CSS**

**JavaScript**

## Как подключить JavaScript к странице?

### 1. Внутренний скрипт (встроенный)

```
<script>

  // JavaScript код здесь

  alert('Привет, мир!');

</script>
```

### 2. Внешний скрипт (рекомендуемый способ)

```
<script src="scripts.js"></script>
```

Лучше размещать перед закрывающим тегом `</body>`, чтобы не блокировать загрузку страницы.

## Основы синтаксиса JavaScript

### 1. Переменные

```
// Старый способ (не рекомендуется)

var oldVariable = "Значение";

// Современные способы

let changeableVar = "Можно изменить";

const constantVar = "Нельзя изменить";
```

### 2. Типы данных

```
// Примитивные типы

const str = "Строка"; // string

const num = 42; // number

const bool = true; // boolean

const nullValue = null; // null

const undefinedValue = undefined; // undefined

const symbol = Symbol('описание'); // symbol
```

```
// Объектные типы

const obj = { key: "value" }; // object

const arr = [1, 2, 3]; // array (особый вид объекта)

const func = function() {}; // function (особый вид объекта)
```

### 3. Операторы

## Содержание

```
// Арифметические

let sum = 5 + 3; // 8

let diff = 5 - 3; // 2


// Сравнения

5 == '5' // true (нестрогое)

5 === '5' // false (строгое)


// Логические

true && false // false (И)

true || false // true (ИЛИ)

!true // false (НЕ)
```

## HTML и CSS

## JavaScript

### 4. Условные конструкции

```
// if-else

if (age > 18) {

  console.log("Взрослый");

} else {

  console.log("Ребёнок");

}


// Тернарный оператор

const status = age > 18 ? "Взрослый" : "Ребёнок";
```

### 5. Циклы

```
// for

for (let i = 0; i < 5; i++) {

  console.log(i);

}


// while

let i = 0;

while (i < 5) {

  console.log(i);

  i++;

}
```

## Работа с DOM (Document Object Model)

DOM — это программный интерфейс для HTML-документов, который представляет страницу в виде дерева объектов.

Основные методы для работы с DOM:

## Выбор элементов

```
// По id
const elem = document.getElementById('elementId');

// По классу
const elems = document.getElementsByClassName('className');

// По тегу
const divs = document.getElementsByTagName('div');

// Современные методы
document.querySelector('.class'); // первый подходящий
document.querySelectorAll('div.class'); // все подходящие
```

## Содержание

### HTML и CSS

### JavaScript

## Изменение содержимого

```
element.textContent = "Новый текст"; // безопасно
element.innerHTML = "<strong>Новый</strong> текст"; // может содержать HTML
```

## Изменение стилей

```
element.style.color = "red";
element.style.fontSize = "20px";
```

## Добавление/удаление классов

```
element.classList.add('new-class');
element.classList.remove('old-class');
element.classList.toggle('toggle-class');
```

## Создание и добавление элементов

```
const newDiv = document.createElement('div');
newDiv.textContent = "Новый элемент";
document.body.appendChild(newDiv);
```

## Обработка событий

События — это действия пользователя или браузера (клик, наведение, загрузка и т.д.)

Основные способы обработки событий:

### Через атрибут HTML (не рекомендуется)

```
<button onclick="alert('Клик!')">Нажми меня</button>
```

### Через свойство элемента

```
const btn = document.querySelector('button');
btn.onclick = function() {
```

```
    alert('Клик!');  
  }  
};
```

## Через addEventListener (рекомендуется)

```
const btn = document.querySelector('button');  
btn.addEventListener('click', function() {  
  alert('Клик!');  
});
```

## Содержание

### HTML и CSS

### JavaScript

## Популярные события:

`click` - **КЛИК** **МЫШЬЮ**

`mouseenter/mouseleave` - **наведение/уход** курсора

`keydown/keyup` - **нажатие/отпускание** клавиши

`submit` - **отправка** формы

`load` - **загрузка** элемента

`scroll` - **прокрутка** страницы

`resize` - **изменение** размера окна